

Universidad Panamericana

Licenciatura en Finanzas

Notebook Documentado de Series de Tiempo

ARIMA, SARIMA, SARIMAX, ACF, PACF, GARCH y
Aplicaciones en Python

Sofia Escamilla

Documento elaborado en formato $\text{\LaTeX}$ con estructura de notebook para
Google Colab

7 de junio de 2026

Índice

1. Objetivo del notebook	3
1.1. Archivos fuente integrados	3
2. Estructura recomendada del repositorio	3
3. Preparación del ambiente en Google Colab	4
4. Descripción de la base de datos	5
5. Fundamentos de Series de Tiempo	6
5.1. Definición	6
5.2. Componentes	6
6. Estacionariedad	7
6.1. Prueba ADF	7
7. ACF y PACF	8
8. Modelos AR, MA, ARMA y ARIMA	8
8.1. Modelo AR(p)	9
8.2. Modelo MA(q)	9
8.3. Modelo ARIMA(p,d,q)	9
9. Modelo SARIMA	10
10. Modelo SARIMAX	11
11. Criterios de selección: AIC, BIC y log-verosimilitud	12
12. Diagnóstico de residuos	12
13. Pronóstico	13
14. Ejercicios de Python: series financieras y ventas simuladas	14
14.1. Serie financiera mensual	14

14.2. Serie mensual simulada con tendencia y estacionalidad	14
15.Caso aplicado: ARIMA y SARIMA sobre AAPL	15
16.Extensión financiera: ARCH y GARCH	16
16.1. GJR-GARCH	17
16.2. EGARCH	18
16.3. GARCH-M	18
17.Value at Risk dinámico	18
18.Metodología Box-Jenkins	19
19.Conclusiones	19
20.Referencias	19
A. Apéndice: checklist para ejecutar en Colab	20

1 Objetivo del notebook

Celda Markdown

Este documento integra y documenta los materiales de **Series de Tiempo** enviados para el proyecto. El objetivo es convertir los documentos teóricos, ejercicios, reportes y base de datos en un notebook ordenado, con celdas tipo Markdown, código Python y desarrollo matemático en $\text{\LaTeX}$.

El notebook está diseñado para ser llevado a **Google Colab**. Por eso se incluyen bloques de instalación, carga de archivos, limpieza de datos, visualización, pruebas de estacionariedad, identificación con ACF/PACF, estimación ARIMA/SARIMA/-SARIMAX y extensiones hacia volatilidad financiera mediante ARCH/GARCH.

1.1 Archivos fuente integrados

Archivo	Uso dentro del notebook
DATABASE_STORE_SALES.csv	Base de datos principal para análisis de ventas por tienda, promoción y días festivos.
README(1).md	Estructura conceptual del repositorio de Series de Tiempo.
Proyecto_ST.tex	y Marco teórico y ejercicios de SARIMAX, GARCH, EGARCH, GJR-GARCH, GARCH-M y VaR.
Proyecto_ST.pdf	
Reporte_Sarima.pdf	
	Documento teórico de ARIMA, SARIMA, SARIMAX, ACF, PACF, AIC, BIC, Ljung-Box y metodología Box-Jenkins.
arima_sarima_aapl.tex	Informe aplicado de modelos ARIMA y SARIMA sobre una serie financiera real.
Preguntas_pyhton.pdf	Ejercicios prácticos en Python sobre descarga de datos, series mensuales, ACF, PACF, tendencia y estacionalidad.

2 Estructura recomendada del repositorio

Celda Markdown

La estructura sugerida del repositorio permite separar datos, notebooks, reportes y documentos fuente. Esta organización facilita subir el proyecto a GitHub y trabajar desde Colab sin mezclar archivos de otras materias.

```
series_de_tiempo/
|
|   datos/
|       DATABASE_STORE_SALES.csv
```

```
|
|     notebooks/
|         notebook_series_tiempo_sofia_escamilla.ipynb
|
|     reportes/
|         Reporte_Sarima.pdf
|         Proyecto_ST.pdf
|         arima_sarima_aapl.pdf
|
|     documentos/
|         Proyecto_ST.tex
|         arima_sarima_aapl.tex
|         Preguntas_pyhton.pdf
|
|     README.md
|     notebook_series_tiempo_sofia_escamilla.tex
```

3 Preparación del ambiente en Google Colab

Celda Markdown

La primera parte del notebook debe preparar las librerías necesarias. Para Series de Tiempo se requieren principalmente pandas, numpy, matplotlib, statsmodels y, para volatilidad financiera, el paquete arch.

```
# =====
# 1. Instalacin de librerias
# =====
!pip install yfinance arch statsmodels --quiet

# =====
# 2. Importacin de paquetes
# =====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.stats.diagnostic import acorr_ljungbox

import warnings
warnings.filterwarnings("ignore")
```

Celda Markdown

En Colab, la base de datos puede cargarse manualmente desde el panel lateral o con `files.upload()`. Para mantener el notebook reproducible, se recomienda conservar el nombre del archivo como `DATABASE_STORE_SALES.csv`.

```
from google.colab import files

# Ejecutar esta celda si el archivo no est cargado en el entorno
# uploaded = files.upload()

df = pd.read_csv("DATABASE_STORE_SALES.csv")
df.head()
```

4 Descripción de la base de datos

Celda Markdown

La base `DATABASE_STORE_SALES.csv` contiene observaciones diarias de ventas por tienda. Las columnas principales son: fecha, tienda, ventas, indicador de promoción e indicador de día festivo.

Métrica	Valor observado
Número de filas	7,300
Número de columnas	5
Columnas	date, store, sales, promo, holiday
Número de tiendas	10
Fecha inicial	2022-01-01
Fecha final	2023-12-31
Venta promedio	228.43
Venta mínima	160.71
Venta máxima	340.73
Proporción de días con pro- moción	20.22
Proporción de días festivos	10.41

```
# Conversin de fecha
df["date"] = pd.to_datetime(df["date"])

# Validacin bsica
print(df.info())
print(df.describe())

# Conteo por tienda
df.groupby("store")["sales"].agg(["count", "mean", "std", "min", "max"])
```

Interpretación financiera / estadística

La base permite trabajar con una estructura de panel: hay varias tiendas observadas durante el tiempo. Para un análisis de series de tiempo puro, se puede agregar la información por fecha para construir una serie total diaria de ventas. También se puede analizar cada tienda por separado.

5 Fundamentos de Series de Tiempo

5.1 Definición

Celda Markdown

Una serie de tiempo es una secuencia de observaciones ordenadas cronológicamente. En finanzas y economía, las series de tiempo se utilizan para analizar precios, ventas, inflación, tasas de interés, tipo de cambio, exportaciones, producción industrial y volatilidad.

Celda LaTeX / Desarrollo matemático

Una serie temporal se puede representar como:

$$\{y_t\}_{t=1}^T$$

donde y_t es el valor observado de la variable en el periodo t , y T es el número total de observaciones.

5.2 Componentes

Celda LaTeX / Desarrollo matemático

De manera general, una serie puede descomponerse como:

$$y_t = T_t + S_t + C_t + \varepsilon_t$$

donde:

- T_t : tendencia.
- S_t : componente estacional.
- C_t : ciclo.
- ε_t : ruido o componente irregular.

Interpretación financiera / estadística

Si una serie presenta tendencia o estacionalidad, generalmente no es estacionaria. En ese caso, antes de ajustar modelos ARIMA o SARIMA se requiere aplicar transformaciones como logaritmos, diferencias regulares o diferencias estacionales.

6 Estacionariedad

Celda Markdown

La estacionariedad es una condición central en la modelación de series de tiempo. Un proceso estacionario en sentido débil mantiene media, varianza y autocovarianza constantes en el tiempo.

Celda LaTeX / Desarrollo matemático

Un proceso $\{y_t\}$ es débilmente estacionario si cumple:

$$E(y_t) = \mu$$

$$Var(y_t) = \sigma^2$$

$$Cov(y_t, y_{t-k}) = \gamma_k$$

para todo t , donde la autocovarianza depende únicamente del rezago k , no del momento específico t .

6.1 Prueba ADF

Celda Markdown

La prueba Augmented Dickey-Fuller (ADF) evalúa si una serie tiene raíz unitaria. La hipótesis nula es que la serie no es estacionaria.

Celda LaTeX / Desarrollo matemático

La hipótesis de la prueba ADF es:

$$H_0 : \text{la serie tiene raíz unitaria}$$

$$H_1 : \text{la serie es estacionaria}$$

Si el $p\text{-value}$ es menor a 0.05, se rechaza H_0 y se concluye que la serie es estacionaria.

```
# Serie total de ventas por fecha
serie_diaria = df.groupby("date")["sales"].sum().asfreq("D")
```



```
# Prueba ADF
resultado_adf = adfuller(serie_diaria.dropna())

print("Estadístico ADF:", resultado_adf[0])
print("p-value:", resultado_adf[1])
print("Valores críticos:", resultado_adf[4])
```

7 ACF y PACF

Celda Markdown

La función de autocorrelación (ACF) mide la correlación entre una serie y sus rezagos. La función de autocorrelación parcial (PACF) mide la relación directa entre y_t y y_{t-k} , eliminando el efecto de los rezagos intermedios.

Celda LaTeX / Desarrollo matemático

La autocorrelación de orden k se define como:

$$\rho_k = \frac{\text{Cov}(y_t, y_{t-k})}{\text{Var}(y_t)} = \frac{\gamma_k}{\gamma_0}$$

```
fig, axes = plt.subplots(2, 1, figsize=(12, 8))

plot_acf(serie_diaria.dropna(), lags=40, ax=axes[0])
axes[0].set_title("ACF de ventas diarias")

plot_pacf(serie_diaria.dropna(), lags=40, ax=axes[1], method="yw")
axes[1].set_title("PACF de ventas diarias")

plt.tight_layout()
plt.show()
```

Interpretación financiera / estadística

En los documentos de teoría se utiliza la ACF para identificar componentes de media móvil $MA(q)$ y la PACF para identificar componentes autorregresivos $AR(p)$. Si hay picos en rezagos estacionales, por ejemplo 7 en series diarias o 12 en series mensuales, puede existir un patrón estacional.

8 Modelos AR, MA, ARMA y ARIMA

8.1 Modelo AR(p)

Celda LaTeX / Desarrollo matemático

Un modelo autorregresivo de orden p , $AR(p)$, se escribe como:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t$$

donde ε_t es ruido blanco.

8.2 Modelo MA(q)

Celda LaTeX / Desarrollo matemático

Un modelo de media móvil de orden q , $MA(q)$, se define como:

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q}$$

8.3 Modelo ARIMA(p,d,q)

Celda LaTeX / Desarrollo matemático

El modelo ARIMA combina autorregresión, diferenciación e innovación de media móvil:

$$\phi_p(B)(1 - B)^d y_t = c + \theta_q(B)\varepsilon_t$$

donde:

$$\phi_p(B) = 1 - \phi_1 B - \cdots - \phi_p B^p$$

$$\theta_q(B) = 1 + \theta_1 B + \cdots + \theta_q B^q$$

```
# Ejemplo: ajuste ARIMA sobre la serie agregada diaria
modelo_arima = ARIMA(serie_diaria.dropna(), order=(1, 1, 1))
resultado_arima = modelo_arima.fit()

print(resultado_arima.summary())
```

Interpretación financiera / estadística

El parámetro d representa el número de diferencias necesarias para volver estacionaria la serie. Si la serie tiene tendencia, suele requerirse $d = 1$. Si ya es estacionaria, puede trabajarse con $d = 0$.

9 Modelo SARIMA

Celda Markdown

El modelo SARIMA extiende ARIMA al incorporar componentes estacionales. Es apropiado cuando la serie tiene patrones que se repiten cada cierto número de periodos, por ejemplo 12 meses en datos mensuales o 7 días en datos diarios.

Celda LaTeX / Desarrollo matemático

La notación general es:

$$SARIMA(p, d, q)(P, D, Q)_s$$

donde:

- p, d, q : componentes no estacionales.
- P, D, Q : componentes estacionales.
- s : longitud del ciclo estacional.

Celda LaTeX / Desarrollo matemático

La formulación con operadores de rezago es:

$$\Phi_P(B^s)\phi_p(B)(1-B)^d(1-B^s)^D y_t = \Theta_Q(B^s)\theta_q(B)\varepsilon_t$$

```
# Ejemplo SARIMA para datos diarios con posible estacionalidad semanal
modelo_sarima = SARIMAX(
    serie_diaria.dropna(),
    order=(1, 1, 1),
    seasonal_order=(1, 0, 1, 7),
    enforce_stationarity=False,
    enforce_invertibility=False
)

resultado_sarima = modelo_sarima.fit(dispatch=False)
print(resultado_sarima.summary())
```

10 Modelo SARIMAX

Celda Markdown

SARIMAX incorpora variables exógenas. En la base de ventas, las variables **promo** y **holiday** pueden utilizarse como regresores externos para explicar cambios en ventas.

Celda LaTeX / Desarrollo matemático

El modelo SARIMAX puede escribirse como:

$$\Phi_P(B^s)\phi_p(B)(1-B)^d(1-B^s)^D y_t = \beta^\top X_t + \Theta_Q(B^s)\theta_q(B)\varepsilon_t$$

donde X_t representa el vector de variables exógenas.

```
# Construcción de serie agregada y variables exgenas
df_diario = df.groupby("date").agg({
    "sales": "sum",
    "promo": "mean",
    "holiday": "max"
}).asfreq("D")

y = df_diario["sales"]
X = df_diario[["promo", "holiday"]]

modelo_sarimax = SARIMAX(
    y,
    exog=X,
    order=(1, 1, 1),
    seasonal_order=(1, 0, 1, 7),
    enforce_stationarity=False,
    enforce_invertibility=False
)

resultado_sarimax = modelo_sarimax.fit(dispatch=False)
print(resultado_sarimax.summary())
```

Interpretación financiera / estadística

En este contexto, un coeficiente positivo de **promo** indicaría que los días con promoción se asocian con mayores ventas, manteniendo constante la dinámica temporal de la serie. Un coeficiente positivo de **holiday** sugeriría que los días festivos incrementan las ventas.

11 Criterios de selección: AIC, BIC y log-verosimilitud

Celda Markdown

Los documentos teóricos enfatizan el principio de parsimonia: elegir el modelo más simple que capture adecuadamente la dinámica de la serie. Para comparar modelos se utilizan AIC y BIC.

Celda LaTeX / Desarrollo matemático

El AIC se define como:

$$AIC = -2 \ln(\hat{L}) + 2k$$

El BIC se define como:

$$BIC = -2 \ln(\hat{L}) + k \ln(n)$$

donde $\hat{L}$ es la verosimilitud maximizada, k es el número de parámetros y n es el tamaño de muestra.

```
print("ARIMA AIC:", resultado_arima.aic)
print("ARIMA BIC:", resultado_arima.bic)

print("SARIMA AIC:", resultado_sarima.aic)
print("SARIMA BIC:", resultado_sarima.bic)

print("SARIMAX AIC:", resultado_sarimax.aic)
print("SARIMAX BIC:", resultado_sarimax.bic)
```

Interpretación financiera / estadística

Un menor AIC o BIC indica mejor balance entre ajuste y complejidad. El BIC penaliza más fuerte la inclusión de parámetros adicionales, por lo que tiende a preferir modelos más conservadores.

12 Diagnóstico de residuos

Celda Markdown

Después de estimar un modelo, los residuos deben comportarse como ruido blanco. Si los residuos presentan autocorrelación, el modelo no capturó toda la estructura temporal.

Celda LaTeX / Desarrollo matemático

La hipótesis nula de la prueba Ljung-Box es:

$$H_0 : \rho_1 = \rho_2 = \dots = \rho_m = 0$$

es decir, no existe autocorrelación significativa en los residuos hasta el rezago m .

```
residuos = resultado_sarimax.resid.dropna()

# Ljung-Box
lb = acorr_ljungbox(residuos, lags=[10, 20], return_df=True)
print(lb)

# ACF y PACF de residuos
fig, axes = plt.subplots(2, 1, figsize=(12, 8))
plot_acf(residuos, lags=40, ax=axes[0])
plot_pacf(residuos, lags=40, ax=axes[1], method="ywm")
plt.tight_layout()
plt.show()
```

Interpretación financiera / estadística

Si el p -value de Ljung-Box es mayor a 0.05, no se rechaza la hipótesis nula de ausencia de autocorrelación residual. Esto sugiere que el modelo es razonablemente adecuado.

13 Pronóstico

Celda Markdown

Una vez validado el modelo, se genera el pronóstico. En SARIMAX, si el modelo usa variables exógenas, también se requieren valores futuros de esas variables.

```
# Horizonte de pronstico
h = 30

# Supuesto simple para variables exgenas futuras:
# promedio historico de promo y holiday
future_exog = pd.DataFrame({
    "promo": [X["promo"].mean()] * h,
    "holiday": [0] * h
})

forecast = resultado_sarimax.get_forecast(steps=h, exog=future_exog)
pred = forecast.predicted_mean
ic = forecast.conf_int()

plt.figure(figsize=(12, 6))
plt.plot(y, label="Historico")
plt.plot(pred, label="Pronstico", color="red")
plt.fill_between(ic.index, ic.iloc[:, 0], ic.iloc[:, 1], alpha=0.2)
plt.title("Pronstico SARIMAX de ventas")
```

```
plt.legend()
plt.show()
```

14 Ejercicios de Python: series financieras y ventas simuladas

Celda Markdown

El archivo Preguntas_pyhton.pdf contiene ejercicios prácticos en Python. En la Pregunta 5 se descarga una serie financiera mensual del S&P 500 con `yfinance`, se grafica la serie y se analizan ACF y PACF. En la Pregunta 6 se simula una serie mensual de ventas con tendencia, estacionalidad y ruido, también analizada con ACF y PACF.

14.1 Serie financiera mensual

```
import yfinance as yf

ticker_symbol = "^GSPC"
start_date = "2000-01-01"
end_date = "2024-01-01"

data = yf.download(
    ticker_symbol,
    start=start_date,
    end=end_date,
    progress=False
)["Close"]

monthly_series = data.resample("MS").first().dropna()
monthly_series.name = f"{ticker_symbol} Cierre Mensual"

plt.figure(figsize=(12, 6))
plt.plot(monthly_series)
plt.title(f"Serie de Cierre Mensual de {ticker_symbol}")
plt.xlabel("Fecha")
plt.ylabel("Precio de cierre")
plt.grid(True)
plt.show()
```

14.2 Serie mensual simulada con tendencia y estacionalidad

```
n_months = 12 * 10
```

```
np.random.seed(45)

dates_monthly = pd.date_range(
    start="2010-01-01",
    periods=n_months,
    freq="MS"
)

trend = np.linspace(500, 1500, n_months)
seasonal_amplitude = 100
seasonal = seasonal_amplitude * np.sin(
    np.linspace(0, 2 * np.pi * (n_months / 12), n_months) + np.pi / 2
)
noise = np.random.normal(0, 30, n_months)

sales_series_simulated = pd.Series(
    trend + seasonal + noise,
    index=dates_monthly,
    name="Ventas Mensuales Simuladas"
)

fig, axes = plt.subplots(3, 1, figsize=(14, 12))
sales_series_simulated.plot(ax=axes[0], title="Serie simulada")
plot_acf(sales_series_simulated, lags=36, ax=axes[1])
plot_pacf(sales_series_simulated, lags=36, ax=axes[2], method="ywm")
plt.tight_layout()
plt.show()
```

Interpretación financiera / estadística

Estos ejercicios son de Series de Tiempo porque se enfocan en periodicidad, tendencia, estacionalidad, ACF y PACF. No pertenecen a Cálculo Estocástico, porque no trabajan con Movimiento Browniano, Lema de Itô, valuación de opciones o simulación neutral al riesgo.

15 Caso aplicado: ARIMA y SARIMA sobre AAPL

Celda Markdown

El archivo `arima_sarima_aapl.tex` desarrolla un informe aplicado a una serie financiera real. Su objetivo es mostrar cómo se construye un modelo ARIMA básico, cómo se compara con otros modelos usando AIC/BIC y cómo se extiende hacia SARIMA cuando existe componente estacional.

Celda LaTeX / Desarrollo matemático

Para una serie financiera P_t , normalmente se trabaja con log-retornos:

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right)$$

Los precios suelen ser no estacionarios, mientras que los rendimientos tienden a ser más cercanos a la estacionariedad.

```
# Esquema general para una accion como AAPL
ticker = "AAPL"
data = yf.download(ticker, start="2020-01-01", end="2024-01-01", progress=False)

prices = data["Close"].dropna()
returns = np.log(prices / prices.shift(1)).dropna()

# ADF en precios y retornos
print("ADF precios:", adfuller(prices)[1])
print("ADF retornos:", adfuller(returns)[1])

# Modelo ARIMA sobre retornos
modelo_aapl = ARIMA(returns, order=(1, 0, 1))
resultado_aapl = modelo_aapl.fit()
print(resultado_aapl.summary())
```

Interpretación financiera / estadística

En series financieras, es común que el precio no sea estacionario, pero los retornos sí lo sean. Por eso la modelación suele hacerse sobre retornos cuando el objetivo es capturar dinámica estadística, riesgo o volatilidad.

16 Extensión financiera: ARCH y GARCH

Celda Markdown

El proyecto de Series de Tiempo también incluye ejercicios sobre ARCH, GARCH, GJR-GARCH, EGARCH y GARCH-M. Estos modelos no explican la media de la serie, sino la varianza condicional, es decir, cómo cambia la volatilidad en el tiempo.

Celda LaTeX / Desarrollo matemático

Un modelo GARCH(1,1) se define como:

$$r_t = \mu + \varepsilon_t$$

$$\varepsilon_t = \sigma_t z_t, \quad z_t \sim N(0, 1)$$

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2$$

Celda LaTeX / Desarrollo matemático

La condición de estacionariedad en covarianza es:

$$\alpha + \beta < 1$$

y la varianza incondicional de largo plazo es:

$$\bar{\sigma}^2 = \frac{\omega}{1 - \alpha - \beta}$$

```
from arch import arch_model

# Modelo GARCH(1,1) sobre retornos porcentuales
returns_pct = returns * 100

garch = arch_model(
    returns_pct,
    vol="GARCH",
    p=1,
    q=1,
    mean="Constant",
    dist="normal"
)

res_garch = garch.fit(dis="off")
print(res_garch.summary())
```

16.1 GJR-GARCH

Celda LaTeX / Desarrollo matemático

El modelo GJR-GARCH incorpora asimetría:

$$\sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2 + \lambda \varepsilon_{t-1}^2 I_{\{\varepsilon_{t-1} < 0\}} + \beta \sigma_{t-1}^2$$

Si $\lambda > 0$, las noticias negativas aumentan más la volatilidad que noticias positivas de igual magnitud.

16.2 EGARCH

Celda LaTeX / Desarrollo matemático

El modelo EGARCH trabaja con el logaritmo de la varianza:

$$\log(\sigma_t^2) = \omega + \beta \log(\sigma_{t-1}^2) + \alpha (|z_{t-1}| - E|z_{t-1}|) + \gamma z_{t-1}$$

Una ventaja es que la varianza siempre es positiva, porque:

$$\sigma_t^2 = \exp(\log(\sigma_t^2)) > 0$$

16.3 GARCH-M

Celda LaTeX / Desarrollo matemático

En un modelo GARCH-M, la volatilidad entra directamente en la media:

$$r_t = \mu + \delta \sigma_t + \varepsilon_t$$

El parámetro δ representa una prima de riesgo dependiente de la volatilidad.

17 Value at Risk dinámico

Celda Markdown

Los ejercicios del proyecto incluyen VaR dinámico con GARCH. La idea es que el riesgo no se mida con una volatilidad constante, sino con una volatilidad condicional actualizada por el modelo.

Celda LaTeX / Desarrollo matemático

Para una posición V , el VaR paramétrico a un día puede escribirse como:

$$VaR_\alpha = -(\mu + q_\alpha \sigma_t)V$$

donde q_α es el cuantil de la distribución utilizada.

Interpretación financiera / estadística

Cuando hay clustering de volatilidad, el VaR basado en GARCH suele ser más sensible a episodios recientes de estrés que el VaR histórico de volatilidad constante. Esto es relevante para gestión de riesgos porque el mercado no mantiene el mismo nivel de volatilidad en todo momento.

18 Metodología Box-Jenkins

Celda Markdown

La metodología Box-Jenkins resume el proceso iterativo para construir modelos ARIMA/SARIMA/SARIMAX. El procedimiento no consiste sólo en estimar un modelo, sino en identificar, estimar, diagnosticar y volver a ajustar si es necesario.

1. **Visualización inicial:** detectar tendencia, estacionalidad, outliers y cambios de régimen.
2. **Transformación:** aplicar logaritmos, diferencias regulares o diferencias estacionales.
3. **Identificación:** usar ACF y PACF para proponer p, q, P, Q .
4. **Estimación:** ajustar varios modelos candidatos.
5. **Selección:** comparar AIC, BIC y log-verosimilitud.
6. **Diagnóstico:** validar residuos con Ljung-Box, ACF/PACF y normalidad.
7. **Pronóstico:** generar predicciones e intervalos de confianza.

19 Conclusiones

Interpretación financiera / estadística

Los documentos integrados muestran que la modelación de Series de Tiempo requiere una secuencia ordenada: primero se comprende la estructura de la serie, después se evalúa estacionariedad, luego se identifican patrones con ACF y PACF, y finalmente se estiman modelos ARIMA, SARIMA o SARIMAX.

La base de ventas permite aplicar este marco a un problema comercial real: pronosticar ventas considerando patrones temporales, promociones y días festivos. Los documentos financieros complementan el análisis al mostrar cómo ARIMA/SARIMA también se aplica a precios y retornos, mientras que GARCH y sus extensiones permiten modelar volatilidad financiera y riesgo dinámico.

En conjunto, el proyecto documenta el uso de Series de Tiempo desde dos perspectivas: una económica/comercial, enfocada en ventas y pronóstico, y otra financiera, enfocada en retornos, volatilidad y riesgo.

20 Referencias

- Box, G. E. P., Jenkins, G. M., Reinsel, G. C. y Ljung, G. M. *Time Series Analysis: Forecasting and Control*.

- Hamilton, J. D. *Time Series Analysis*.
- Hyndman, R. J. y Athanasopoulos, G. *Forecasting: Principles and Practice*.
- Tsay, R. S. *Analysis of Financial Time Series*.
- Enders, W. *Applied Econometric Time Series*.
- Documentos fuente proporcionados: `Reporte_Sarima.pdf`, `Proyecto_ST.tex`, `Proyecto_ST.pdf`, `arima_sarima_aapl.tex`, `Preguntas_pyhton.pdf`, `DATABASE_STORE_SALES.csv`, `README(1).md`.

A Apéndice: checklist para ejecutar en Colab

1. Subir `DATABASE_STORE_SALES.csv`.
2. Instalar `yfinance` y `arch`.
3. Importar librerías.
4. Leer y limpiar la base.
5. Agregar ventas por fecha.
6. Graficar la serie.
7. Aplicar prueba ADF.
8. Revisar ACF/PACF.
9. Estimar ARIMA.
10. Estimar SARIMA.
11. Estimar SARIMAX con `promo` y `holiday`.
12. Diagnosticar residuos.
13. Generar pronóstico.
14. Para series financieras, descargar AAPL o S&P 500 con `yfinance`.
15. Estimar modelos GARCH si el objetivo es volatilidad.